

UNMATCHED PERFORMANCE

NVIDIA RTX PRO™ 6000 BLACKWELL SERVER EDITION

PNY

NVIDIA

[Explore features](#)

A Gentle Introduction to Markov Chain Monte Carlo for Probability

by Jason Brownlee on [September 25, 2019](#) in [Probability](#) [15](#)

[Share](#)

[Share](#)

Post

Probabilistic inference involves estimating an expected value or density using a probabilistic model.

Often, directly inferring values is not tractable with probabilistic models, and instead, approximation methods must be used.

Markov Chain Monte Carlo sampling provides a class of algorithms for systematic random sampling from high-dimensional probability distributions. Unlike Monte Carlo sampling methods that are able to draw independent samples from the distribution, Markov Chain Monte Carlo methods draw samples where the next sample is dependent on the existing sample, called a Markov Chain. This allows the algorithms to narrow in on the quantity that is being approximated from the distribution, even with a large number of random variables.

In this post, you will discover a gentle introduction to Markov Chain Monte Carlo for machine learning.

After reading this post, you will know:

- Monte Carlo sampling is not effective and may be intractable for high-dimensional probabilistic models.
- Markov Chain Monte Carlo provides an alternate approach to random sampling a high-dimensional probability distribution where the next sample is dependent upon the current sample.
- Gibbs Sampling and the more general Metropolis-Hastings algorithm are the two most common approaches to Markov Chain Monte Carlo sampling.

Kick-start your project with my new book [Probability for Machine Learning](#), including *step-by-step tutorials* and the *Python source code* files for all examples.

Let's get started.



A Gentle Introduction to Markov Chain Monte Carlo for Probability
Photo by [Murray Foubister](#), some rights reserved.

Overview

This tutorial is divided into three parts; they are:

1. Challenge of Probabilistic Inference
2. What Is Markov Chain Monte Carlo
3. Markov Chain Monte Carlo Algorithms

Challenge of Probabilistic Inference

Calculating a quantity from a probabilistic model is referred to more generally as probabilistic inference, or simply inference.

For example, we may be interested in calculating an expected probability, estimating the density, or other properties of the probability distribution. This is the goal of the probabilistic model, and the name of the inference performed often takes on the name of the probabilistic model, e.g. Bayesian Inference is performed with a Bayesian probabilistic model.

The direct calculation of the desired quantity from a model of interest is intractable for all but the most trivial probabilistic models. Instead, the expected probability or density must be approximated by other means.



For most probabilistic models of practical interest, exact inference is intractable, and so we have to resort to some form of approximation.

— Page 523, [Pattern Recognition and Machine Learning](#), 2006.

The desired calculation is typically a sum of a discrete distribution of many random variables or integral of a continuous distribution of many variables and is intractable to calculate. This problem exists in both schools of probability, although is perhaps more prevalent or common with Bayesian probability and integrating over a posterior distribution for a model.



Bayesians, and sometimes also frequentists, need to integrate over possibly high-dimensional probability distributions to make inference about model parameters or to make predictions. Bayesians need to integrate over the posterior distribution of model parameters given the data, and frequentists may need to integrate over the distribution of observables given parameter values.

— Page 1, [Markov Chain Monte Carlo in Practice](#), 1996.

The typical solution is to draw independent samples from the probability distribution, then repeat this process many times to approximate the desired quantity. This is referred to as Monte Carlo sampling or Monte Carlo integration, named for the city in Monaco that has many casinos.

The problem with Monte Carlo sampling is that it does not work well in high-dimensions. This is firstly because of the curse of dimensionality, where the volume of the sample space increases exponentially with the number of parameters (dimensions).

Secondly, and perhaps most critically, this is because Monte Carlo sampling assumes that each random sample drawn from the target distribution is independent and can be independently drawn. This is typically not the case or intractable for inference with Bayesian structured or graphical probabilistic models.

Want to Learn Probability for Machine Learning

Take my free 7-day email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

What Is Markov Chain Monte Carlo

The solution to sampling probability distributions in high-dimensions is to use [Markov Chain Monte Carlo](#), or MCMC for short.



The most popular method for sampling from high-dimensional distributions is Markov chain Monte Carlo or MCMC

— Page 837, [Machine Learning: A Probabilistic Perspective](#), 2012.

Like Monte Carlo methods, Markov Chain Monte Carlo was first developed around the same time as the development of the first computers and was used in calculations for particle physics required as part of the Manhattan project for developing the atomic bomb.

Monte Carlo

[Monte Carlo](#) is a technique for randomly sampling a probability distribution and approximating a desired quantity.



Monte Carlo algorithms, [...] are used in many branches of science to estimate quantities that are difficult to calculate exactly.

— Page 530, [Artificial Intelligence: A Modern Approach](#), 3rd edition, 2009.

Monte Carlo methods typically assume that we can efficiently draw samples from the target distribution. From the samples that are drawn, we can then estimate the sum or integral quantity as the mean or variance of the drawn samples.

A useful way to think about a Monte Carlo sampling process is to consider a complex two-dimensional shape, such as a spiral. We cannot easily define a function to describe the spiral, but we may be able to draw samples from the domain and determine if they are part of the spiral or not. Together, a large number of samples drawn from the domain will allow us to summarize the shape (probability density) of the spiral.

Markov Chain

[Markov chain](#) is a systematic method for generating a sequence of random variables where the current value is probabilistically dependent on the value of the prior variable. Specifically, selecting the next variable is only dependent upon the last variable in the chain.



A Markov chain is a special type of stochastic process, which deals with characterization of sequences of random variables. Special interest is paid to the dynamic and the limiting behaviors of the sequence.

Consider a board game that involves rolling dice, such as [snakes and ladders](#) (or chutes and ladders). The roll of a die has a uniform probability distribution across 6 stages (integers 1 to 6). You have a position on the board, but your next position on the board is only based on the current position and the random roll of the dice. Your specific positions on the board form a Markov chain.

Another example of a Markov chain is a random walk in one dimension, where the possible moves are 1, -1, chosen with equal probability, and the next point on the number line in the walk is only dependent upon the current position and the randomly chosen move.



At a high level, a Markov chain is defined in terms of a graph of states over which the sampling algorithm takes a random walk.

— Page 507, [Probabilistic Graphical Models: Principles and Techniques](#), 2009.

Markov Chain Monte Carlo

Combining these two methods, Markov Chain and Monte Carlo, allows random sampling of high-dimensional probability distributions that honors the probabilistic dependence between samples by constructing a Markov Chain that comprise the Monte Carlo sample.



MCMC is essentially Monte Carlo integration using Markov chains. [...] Monte Carlo integration draws samples from the the required distribution, and then forms sample averages to approximate expectations. Markov chain Monte Carlo draws these samples by running a cleverly constructed Markov chain for a long time.

— Page 1, [Markov Chain Monte Carlo in Practice](#), 1996.

Specifically, MCMC is for performing inference (e.g. estimating a quantity or a density) for probability distributions where independent samples from the distribution cannot be drawn, or cannot be drawn easily.

As such, Monte Carlo sampling cannot be used.

Instead, samples are drawn from the probability distribution by constructing a Markov Chain, where the next sample that is drawn from the probability distribution is dependent upon the last sample that was drawn. The idea is that the chain will settle on (find equilibrium) on the desired quantity we are inferring.

Yet, we are still sampling from the target probability distribution with the goal of approximating a desired quantity, so it is appropriate to refer to the resulting collection of samples as a Monte Carlo sample, e.g. extent of samples drawn often forms one long Markov chain.

The idea of imposing a dependency between samples may seem odd at first, but may make more sense if we consider domains like the [random walk](#) or snakes and ladders games, where such dependency between samples is required.

Markov Chain Monte Carlo Algorithms

There are many Markov Chain Monte Carlo algorithms that mostly define different ways of constructing the Markov Chain when performing each Monte Carlo sample.

The random walk provides a good metaphor for the construction of the Markov chain of samples, yet it is very inefficient. Consider the case where we may want to calculate the expected probability; it is more efficient to zoom in on that quantity or density, rather than wander around the domain. Markov Chain Monte Carlo algorithms are attempts at carefully harnessing properties of the problem in order to construct the chain efficiently.



This sequence is constructed so that, although the first sample may be generated from the prior, successive samples are generated from distributions that provably get closer and closer to the desired posterior.

— Page 505, [Probabilistic Graphical Models: Principles and Techniques](#), 2009.

MCMC algorithms are sensitive to their starting point, and often require a warm-up phase or burn-in phase to move in towards a fruitful part of the search space, after which prior samples can be discarded and useful samples can be collected.

Additionally, it can be challenging to know whether a chain has converged and collected a sufficient number of steps. Often a very large number of samples are required and a run is stopped given a fixed number of steps.



... it is necessary to discard some of the initial samples until the Markov chain has burned in, or entered its stationary distribution.

— Page 838, [Machine Learning: A Probabilistic Perspective](#), 2012.

The most common general Markov Chain Monte Carlo algorithm is called Gibbs Sampling; a more general version of this sampler is called the Metropolis-Hastings algorithm.

Let's take a closer look at both methods.

Gibbs Sampling Algorithm

The [Gibbs Sampling](#) algorithm is an approach to constructing a Markov chain where the probability of the next sample is calculated as the conditional probability given the prior sample.

Samples are constructed by changing one random variable at a time, meaning that subsequent samples are very close in the search space, e.g. local. As such, there is some risk of the chain getting stuck.



The idea behind Gibbs sampling is that we sample each variable in turn, conditioned on the values of all the other variables in the distribution.

— Page 838, [Machine Learning: A Probabilistic Perspective](#), 2012.

Gibbs Sampling is appropriate for those probabilistic models where this conditional probability can be calculated, e.g. the distribution is discrete rather than continuous.



... Gibbs sampling is applicable only in certain circumstances; in particular, we must be able to sample from the distribution $P(X_i | x_{-i})$. Although this sampling step is easy for discrete graphical models, in continuous models, the conditional distribution may not be one that has a parametric form that allows sampling, so that Gibbs is not applicable.

— Page 515, [Probabilistic Graphical Models: Principles and Techniques](#), 2009.

Metropolis-Hastings Algorithm

The [Metropolis-Hastings Algorithm](#) is appropriate for those probabilistic models where we cannot directly sample the so-called next state probability distribution, such as the conditional probability distribution used by Gibbs Sampling.



Unlike the Gibbs chain, the algorithm does not assume that we can generate next-state samples from a particular target distribution.

— Page 517, [Probabilistic Graphical Models: Principles and Techniques](#), 2009.

Instead, the Metropolis-Hastings algorithm involves using a surrogate or proposal probability distribution that is sampled (sometimes called the kernel), then an acceptance criterion that decides whether the new sample is accepted into the chain or discarded.



They are based on a Markov chain whose dependence on the predecessor is split into two parts: a proposal and an acceptance of the proposal. The proposals suggest an arbitrary next step in the trajectory of the chain and the acceptance makes sure the appropriate limiting direction is maintained by rejecting unwanted moves of the chain.

— Page 6, [Markov Chain Monte Carlo: Stochastic Simulation for Bayesian Inference](#), 2006.

The acceptance criterion is probabilistic based on how likely the proposal distribution differs from the true next-state probability distribution.

The Metropolis-Hastings Algorithm is a more general and flexible Markov Chain Monte Carlo algorithm, subsuming many other methods.

For example, if the next-step conditional probability distribution is used as the proposal distribution, then the Metropolis-Hastings is generally equivalent to the Gibbs Sampling Algorithm. If a symmetric proposal distribution is used like a Gaussian, the algorithm is equivalent to another MCMC method called the Metropolis algorithm.

Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Books

- [Markov Chain Monte Carlo in Practice](#), 1996.
- [Markov Chain Monte Carlo: Stochastic Simulation for Bayesian Inference](#), 2006.
- [Handbook of Markov Chain Monte Carlo](#), 2011.
- [Probabilistic Graphical Models: Principles and Techniques](#), 2009.

Chapters

- Chapter 24 Markov chain Monte Carlo (MCMC) inference, [Machine Learning: A Probabilistic Perspective](#), 2012.
- Section 11.2. Markov Chain Monte Carlo, [Pattern Recognition and Machine Learning](#), 2006.
- Section 17.3 Markov Chain Monte Carlo Methods, [Deep Learning](#), 2016.

Articles

- [Monte Carlo method](#), Wikipedia.
- [Markov chain](#), Wikipedia.
- [Markov chain Monte Carlo](#), Wikipedia.
- [Gibbs sampling](#), Wikipedia.
- [Metropolis–Hastings algorithm](#), Wikipedia.
- [MCMC sampling for dummies](#), 2015.
- [How would you explain Markov Chain Monte Carlo \(MCMC\) to a layperson?](#)

Summary

In this post, you discovered a gentle introduction to Markov Chain Monte Carlo for machine learning.

Specifically, you learned:

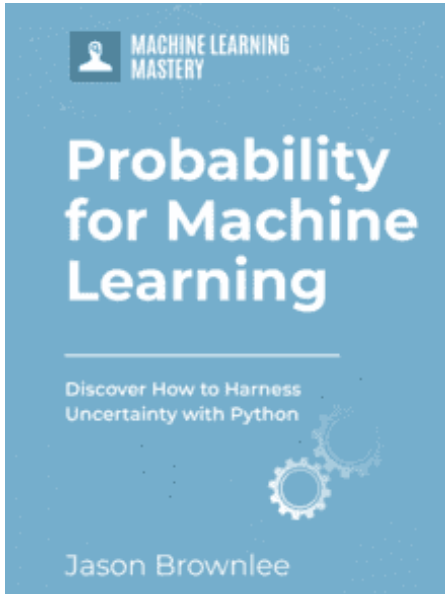
- Monte Carlo sampling is not effective and may be intractable for high-dimensional probabilistic models.
- Markov Chain Monte Carlo provides an alternate approach to random sampling a high-dimensional probability distribution where the next sample is dependent upon the current sample.
- Gibbs Sampling and the more general Metropolis-Hastings algorithm are the two most

common approaches to Markov Chain Monte Carlo sampling.

Do you have any questions?

Ask your questions in the comments below and I will do my best to answer.

Get a Handle on Probability for Machine Learning!



Develop Your Understanding of Probability

...with just a few lines of python code

Discover how in my new Ebook:

[Probability for Machine Learning](#)

It provides **self-study tutorials** and **end-to-end projects** on:
Bayes Theorem, Bayesian Optimization, Distributions, Maximum Likelihood, Cross-Entropy, Calibrating Models
and much more...

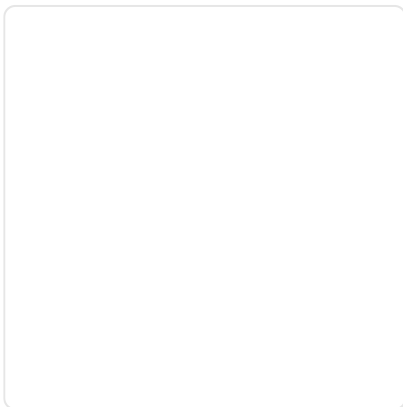
Finally Harness Uncertainty in Your Projects

Skip the Academics. Just Results.

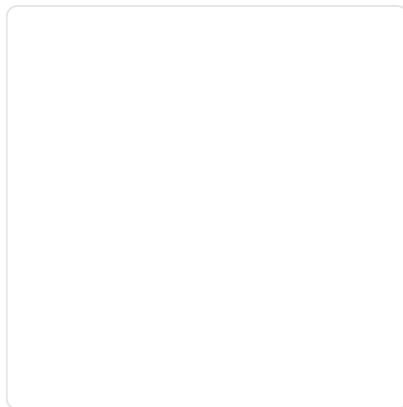
SEE WHAT'S INSIDE

Post

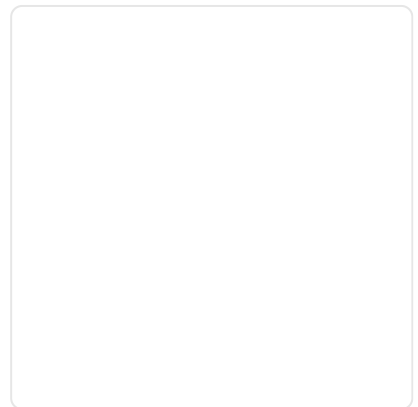
More On This Topic



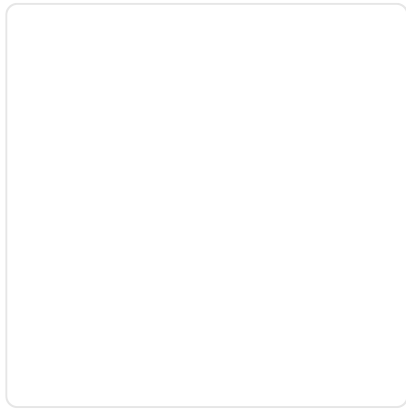
[A Gentle Introduction to Monte Carlo Sampling for...](#)



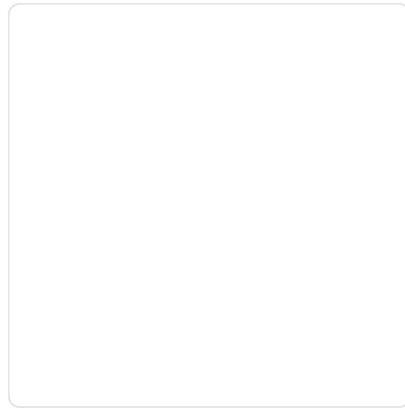
[The Chain Rule of Calculus for Univariate and...](#)



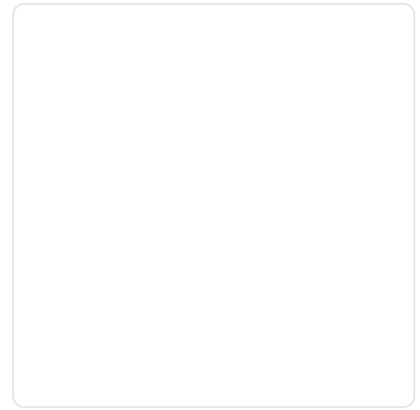
[The Chain Rule of Calculus - Even More Functions](#)



[A Gentle Introduction to Probability Scoring Methods...](#)



[A Gentle Introduction to Probability Distributions](#)



[A Gentle Introduction to Probability Density Estimation](#)



About Jason Brownlee

Jason Brownlee, PhD is a machine learning specialist who teaches developers how to get results with modern machine learning methods via hands-on tutorials.

[View all posts by Jason Brownlee](#) →

[< A Gentle Introduction to Monte Carlo Sampling for Probability](#)

[A Gentle Introduction to Maximum a Posteriori \(MAP\) for Machine Learning >](#)

15 Responses to *A Gentle Introduction to Markov Chain Monte Carlo for Probability*

marco November 7, 2019 at 4:57 am <#>

REPLY ↩

Hello Jason,
I have a question. What is a gradient in very easy words? Do you have an easy example?
Thanks Marco

Jason Brownlee November 7, 2019 at 6:48 am <#>

REPLY ↩

A gradient is a slope at a point on a function:

Raj November 10, 2019 at 5:31 am #

REPLY ↩

Once again thanks for your post in simple language. Would like to learn more about applications of MCMC. Would you please share some insights?

Jason Brownlee November 10, 2019 at 8:27 am #

REPLY ↩

Yes, I hope to cover the topic in a future book.

quan chen April 19, 2020 at 6:04 pm #

REPLY ↩

no sample

Jason Brownlee April 20, 2020 at 5:25 am #

REPLY ↩

What do you mean?

yk x July 10, 2020 at 4:13 pm #

REPLY ↩

Good elaboration with clear motivation, vivid examples to help me understand.

Jason Brownlee July 11, 2020 at 6:01 am #

REPLY ↩

Thanks.

Hargun Oberoi February 25, 2021 at 11:49 pm #

REPLY ↩

Hi Jason!

I've been (quietly) following your posts and it has been a great help for my research.

Recently, I am trying to build and train a basic neural network using MCMC.

It is for educational purpose only and I understand that it's not an effective method for deeper networks.

I could successfully use TensorflowProbability to perform a linear regression using the example in this post. https://juanitorduz.github.io/tfp_lm/

However, I'm not making any progress on how to do it for a neural network. Do you know any good resources on this?

Thanks for making neural nets easier one post at a time.

Jason Brownlee February 26, 2021 at 4:59 am <#>

REPLY 

Well done.

Sorry, I don't have any good resources for this.

Jack June 16, 2021 at 6:34 pm <#>

REPLY 

Hello Jason,

could you give me some example about Markov Chain ?

Jason Brownlee June 17, 2021 at 6:15 am <#>

REPLY 

Thanks for the suggestion.

Franco Arda July 2, 2021 at 4:36 pm <#>

REPLY 

Financial products which are path-dependent and multi-dimensional (e.g., interest rates, volatility, dividends ...).

M.A September 16, 2021 at 5:38 pm <#>

REPLY 

Hi, I also suggest referring to the books: "Statistical Rethinking, A Bayesian Course with Examples in R and Stan" by McElreath and "Bayesian Data Analysis" by Gelman. They give a contemporary view of Bayesian analysis and MCMC (where they use the Hamiltonian MC algorithm, a superior to Gibbs and Metropolis.)



Adrian Tam September 17, 2021 at 12:00 am <#>

REPLY 

Thanks for sharing

Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT

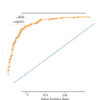


Welcome!
I'm *Jason Brownlee* PhD
and I **help developers** get results with **machine learning**.
[Read more](#)

Never miss a tutorial:



Picked for you:



[How to Use ROC Curves and Precision-Recall Curves for Classification in Python](#)



[How and When to Use a Calibrated Classification Model with scikit-learn](#)



[How to Implement Bayesian Optimization from Scratch in Python](#)



[Probability for Machine Learning \(7-Day Mini-Course\)](#)



[How to Calculate the KL Divergence for Machine Learning](#)

Loving the Tutorials?

The [Probability for Machine Learning](#) EBook is where you'll find the ***Really Good*** stuff.

>> SEE WHAT'S INSIDE

ing Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out
t our [corporate website](#) to learn more about our mission and team.

CLAIMER TERMS CONTACT SITEMAP ADVERTISE WITH US

g Tech Media All Rights Reserved